
Kopibara RLab Agent: Autonomous Code Search for KuaiRand Ranking

Phanuphat Srisukhawasu

phanuphat.srisukhawasu@gmail.com
Kopibara RLab

Supachod Trakansirorut

spchdt@gmail.com
Kopibara RLab

September 1, 2026

Abstract

This work implements an autonomous machine-learning research agent for the KuaiRand ranking task. The controller treats model development as a bounded search over exact code edits: an OpenAI language model proposes a hypothesis, the candidate runs on train and validation data, and the controller retains or rejects the change from the measured primary score. A leakage-safe LightGBM ranker with lagged multi-feedback history features is used as the seed. The final Pure run reaches validation primary 0.6299, while the larger 1K run reaches 0.7498 after 21 logged trials. Both runs produce schema-checked submissions without manual intervention.

Keywords autonomous machine learning · recommender systems · code search

1 Objective and evaluation

The task is to automate the engineering loop for a recommendation ranker: inspect the benchmark contract, construct features, train and tune a model, evaluate on public validation data, and revise the code. KuaiRand ranks logged impressions within each user. The relevance label is `long_view`; the primary score is the mean of GAUC and nDCG@5. KuaiRand-Pure is the required benchmark and KuaiRand-1K is the larger bonus variant.

The controller develops against train and validation splits only. Test rows are available for final prediction, but their labels are removed before candidate code runs and are never used for model selection. The organizer’s convergence rule is an improvement of more than 0.002 over three consecutive iterations; the run also has a 50-iteration and six-hour backstop.

2 Agent design

The implementation follows the code-search framing of [AIDE](#). The planner receives the benchmark contract and the measured solution tree, then returns one hypothesis and a small set of exact-match replacements. The runner compiles the candidate, executes it in a bounded subprocess, parses validation metrics, and records the code diff, resource use, and recovery events. The controller selects the highest measured node as the next parent. A failed candidate can be repaired once; a second failure is retained as a `recovered_failure` node and cannot become a parent.

The seed candidate is a grouped LightGBM ranker. It combines known context fields with four leakage-safe history scopes: user, video, author, and user-video. For every scope, the feature builder records prior interaction count, recency, feedback rates, cumulative feedback, and elapsed time since the

previous observation. The history state is updated chronologically, so a row cannot use its own or a later outcome. All available feedback signals are used as auxiliary history features while `long_view` remains the scored label.

3 Search results

The validation results selected from the two autonomous runs are summarized in Table 1. The 1K reference is a measured FM runtime baseline because the starter kit publishes an official reference only for Pure.

Benchmark	Reference primary	GAUC	nDCG@5	Primary
KuaiRand-Pure	0.6016 official	0.7059	0.5538	0.6299
KuaiRand-1K	0.6079 measured	0.7107	0.7888	0.7498

Table 1: Validation results. The 1K reference is not an organizer-published score.

The Pure seed already exceeds the official validation primary by 0.0283. Its three child trials tested the `rank_xendcg` objective, a wider ranking truncation, and disabled query normalization; none improved the seed. The final Pure submission is therefore selected after three search iterations.

On 1K, the initial candidate scored below the measured FM reference at 0.6038. The first substantial gain came from allowing `rank_xendcg` to operate on grouped queries, raising primary to 0.6920. Increasing tree capacity to 63 leaves raised it to 0.7311. Adding `lambda_l2=2.0` raised it to 0.7487, and increasing the histogram resolution to `max_bin=767` produced the run's highest primary, 0.7498. Nearby values were tested: `max_bin=511` reached 0.7493, 1023 reached 0.7473, and 640 reached 0.7475.

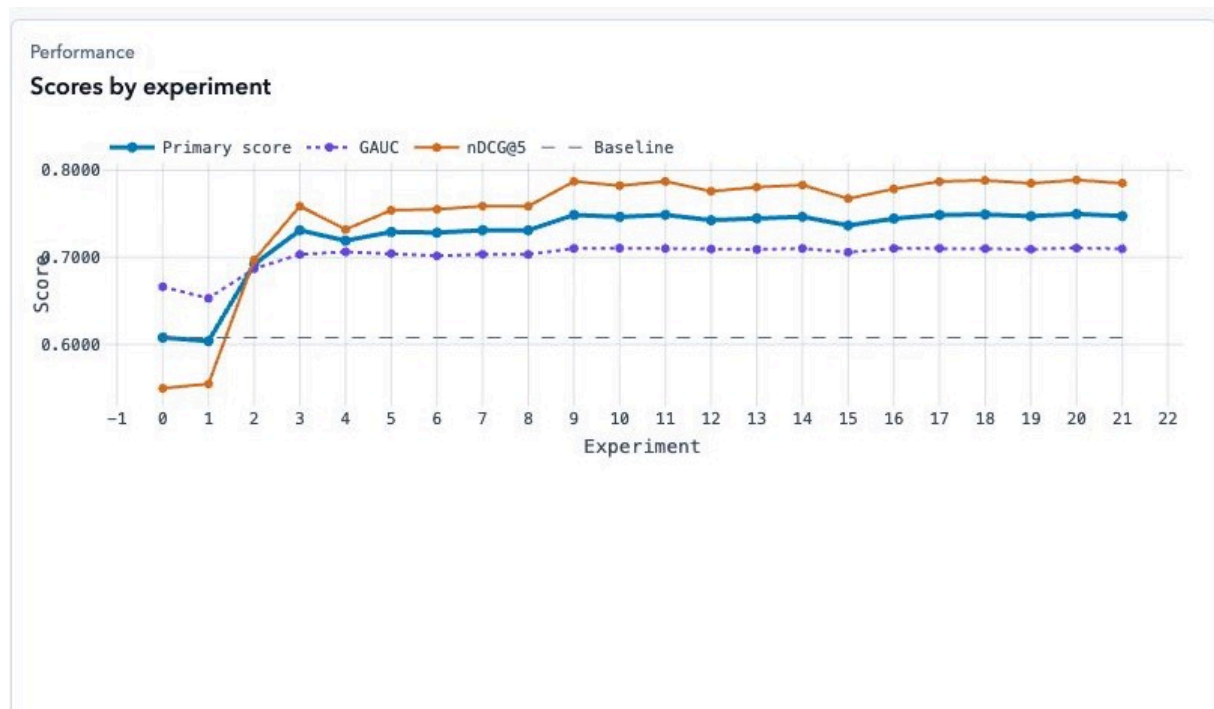


Figure 1: 1K validation trajectory captured from the local dashboard. The plotted lines show primary, GAUC, nDCG@5, and the measured reference.

The trajectory in Figure 1 shows the same pattern as the run log: the largest gains came from changing the ranking objective and tree capacity, while later edits refined regularization and histogram resolution. The selected 1K node is 019-child; the final candidate after it was a recorded failure and did not replace the checkpoint.

4 Reproducibility and scope

The run logs preserve each hypothesis, exact diff, validation metrics, subprocess command, token count, and recovery event. The final CSVs passed the starter-kit's alignment checker: 170,588 rows for Pure and 4,132,081 rows for 1K. Both manifests record zero manual interventions and `hidden_test_access: false`.

The reported numbers are public-validation results. Hidden-test scores are unavailable until organizer evaluation, so no hidden-test improvement is inferred. The 1K result is also reported without an official delta because no official 1K baseline is included in the starter kit.

APPENDIX A Run record

Benchmark	Run ID	Iterations	Wall-clock	LLM tokens	Manual
KuaiRand-Pure	20260831T111545Z	3	5m 5s	12,901	0
KuaiRand-1K	20260831T124656Z	21	6h 7m	115,212	0

The source implementation, manifests, and row-aligned submissions are stored in the repository. The method is compatible with the organizer’s date-based split contract and uses no external training data or benchmark test labels.

APPENDIX B References

Bibliography